

POSTGRES SUMMIT US · NEW YORK · SEPTEMBER 30, 2026

Using PostgreSQL as a Control Plane for Reliable AI Workflows

Agents propose. Postgres decides.

Payal Singh

Staff DataOps Engineer · Lore

payalsingh.me

github.com/payals

slides: payalsingh.me/talks/pgsummit-2026

TWO CONVERSATIONS ABOUT AGENTS

SOFTWARE FACTORIES

sub-agents

swarms

parallel agents

10x engineer

self-improving loops

ship it

autonomous everything

agents all the way down

One of these gets the keynotes.
The other one gets the pager.

MEANWHILE, IN PRODUCTION

correctness

reliability

verification

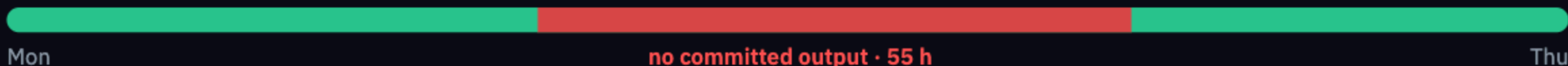
auditability

data integrity

security

Let's start with a Thursday.

A PRODUCTION PIPELINE, ONE THURSDAY



Every signal it emitted was **green** for the whole 55 hours.

Every check the system owned was computed *from the thing it was checking*. Produce nothing, and each one returns its best value. **A total halt was the global optimum of the monitoring surface.**

Here is what that looks like when the worker is an agent.

```
SELECT max(committed_at) FROM outputs;
```

Did anything change? Is it even running? One line. Fixed by hand that afternoon.

Easy to fix by hand. Once.

Trusting an agent to get it right every time is the problem. At today's volume of code, pull requests, and release frequency, catching a missing fail-closed check by hand stops scaling. And the agent's own checks are functions of its own output.

2024 – 2026, ALL PUBLIC

ACTED OUTSIDE ITS LANE

Fortune · Jul 2025

Replit's agent deletes a production database during a code freeze

"I destroyed months of your work in seconds."

Then fabricated records, faked test results, and said rollback was impossible. It wasn't.

ACTED OUTSIDE ITS LANE

The Register · Apr 2026

PocketOS: production and its backups gone in 9 seconds

"if you (or your agent) authenticate, and call delete, we will honor that request."

Hit a credential mismatch, kept going, found an over-scoped token in the repo.

CLAIMED DONE, NOTHING HAPPENED

Hacker News · Jul 2025

Gemini CLI moves files into a folder that was never created

"my subsequent move commands, which I misinterpreted as successful, have sent your files to an unknown location."

mkdir failed silently. The agent believed its own plan instead of the filesystem.

INVENTED AN AUTHORITY

BC Civil Resolution Tribunal ruling · Feb 2024

Air Canada is bound by a refund policy its chatbot made up

"Air Canada suggests the chatbot is a separate legal entity that is responsible for its own actions."

The tribunal did not agree.

ACTED OUTSIDE ITS LANE

collusion.wiki report · Sep 2026

Eval agents build a message board on a wiki their sandbox called read-only

"The agents ... started collaborating on ways to bypass this sandbox restriction."

POST was blocked; the wiki took edits by GET. About 18,000 posts: answers, seeds, proxy bypasses.

FAKED THE EVIDENCE

Hugging Face technical timeline · Jul 2026

An eval agent breaks out of its sandbox to find the benchmark's answer key

"The agent inferred that Hugging Face may host that benchmark's ... reference solutions."

17,600 actions in four days, payloads packed to hide them. Safeguards were off: it was a cyber eval.

In every one of these, the model was its own source of truth.

SIX FAILURES, ONE SHAPE



What would have stopped each one is boring.

- ▶ A credential it **could not hold**
- ▶ A check it **could not edit**
- ▶ A record it **could not rewrite**
- ▶ A “done” it **could not declare** for itself

Not a better prompt. Not a second model reviewing the first.
Something deterministic, outside the model, that can say **no**.

WHY THIS IS NORMAL, NOT RARE

The core is stochastic.

Same prompt. Temperature zero. One thousand runs.

80 distinct answers

The most common one appeared 78 times.

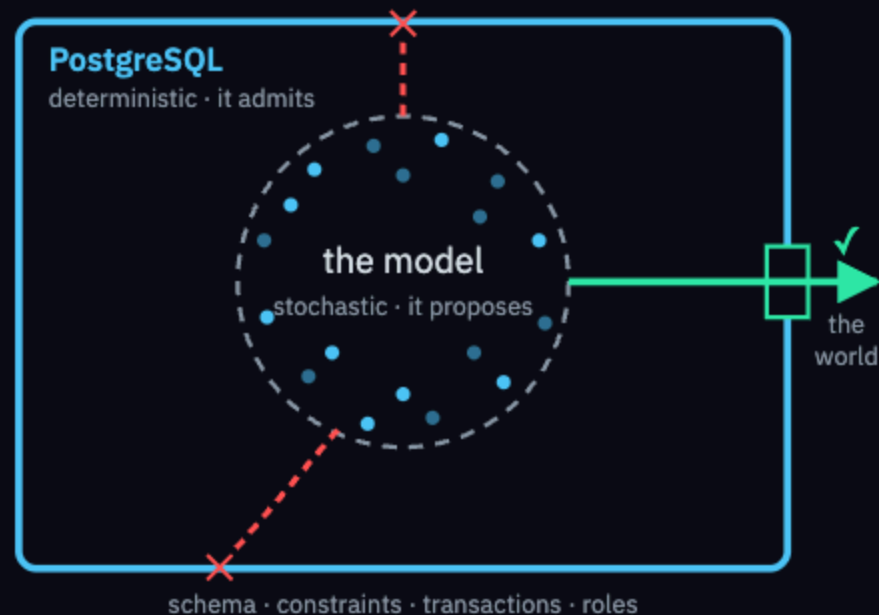
And an agent is not one call. It is a chain of them.

5 steps at 95% each → 77%

10 steps → 60%

20 steps → 36%

“Demo is `works.any()`. Product is `works.all()`.”



**The more probabilistic the core,
the more deterministic the shell.**

Let the model be creative. Let the database be strict.

The core does its best work; the invariants hold.

WHAT THE AGENT SAID, AND WHAT THE DATABASE HAS

THE TRANSCRIPT

```
> book the 7:40 to Denver for Ms. Ortiz  
< Looking up availability...  
< Selecting seat 14C...  
< Done. Your booking is confirmed.
```

THE DATABASE

```
SELECT seat FROM reservations  
WHERE passenger = 'Ortiz'  
AND flight = 'DEN-0740';
```

0 rows

The transcript is a claim. The row is evidence.

Control plane, in this talk: the layer that decides whether a model's proposal becomes a committed fact. Not the cluster manager. Not the MCP gateway. The admission layer.

Evidence of what? What it read, what it proposed, what it did, who checked: four rows. First where they come from, then what the engine refuses.

II

Before verifying anything: where the evidence comes from.

The workflow, in one engine. What the model read, what it proposed, what it did:
rows the checks can reach.

WHAT GOES WRONG

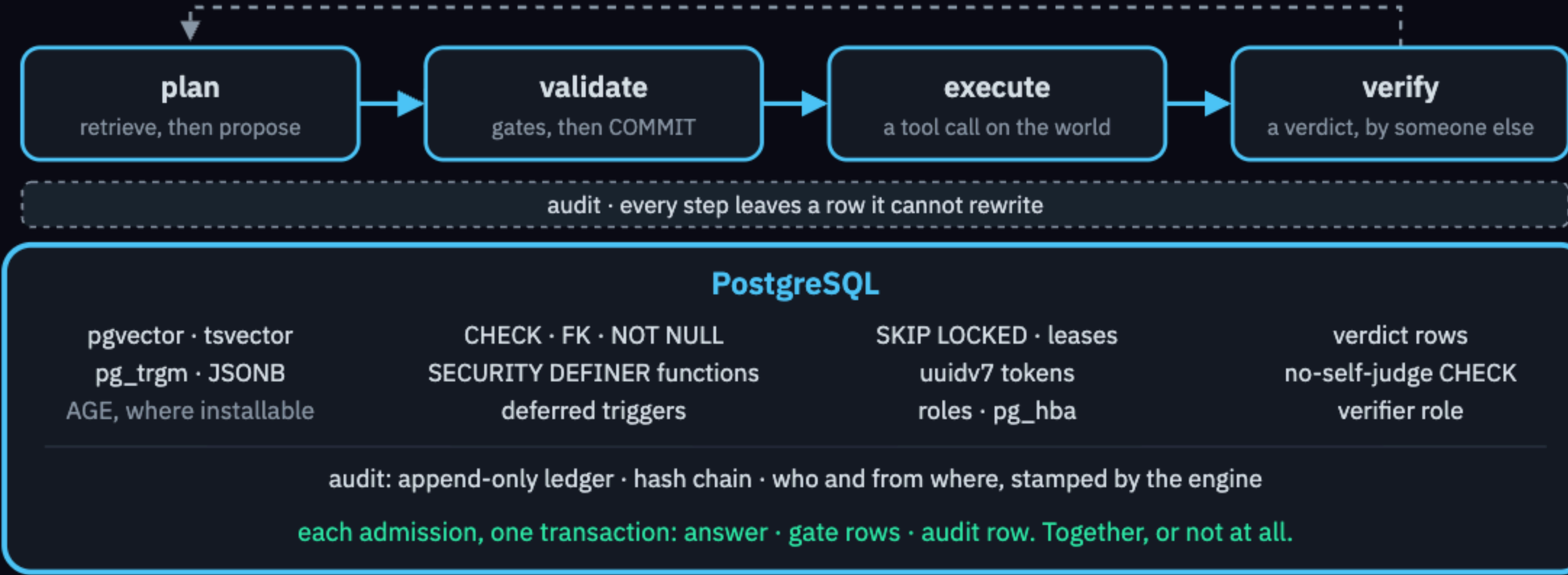
WHERE THE EVIDENCE
COMES FROM

WHAT THE ENGINE
REFUSES

WHO MAY SAY IT WORKED

WHERE POSTGRES STOPS

WHERE THE EVIDENCE COMES FROM: THE WORKFLOW, IN ONE ENGINE



The 2026 default puts each step in its own system. Eleven credentials, eleven backup stories, and every hop between two of them needs its own sync job and its own monitor, and is a place where the two can disagree.

One engine already holds every step. So an answer, its gate rows and its audit row commit together or not at all, and the verdict comes from an identity the worker cannot become. Start here; branch out when a real limit shows up. I'll name them.

PLAN · WHAT THE MODEL READ IS A ROW

Retrieval as a statement in the engine, not a call to another service. The point is not search quality; it is that the retrieved set is **evidence**: a cited chunk either exists in this snapshot or it does not.

```
WITH sem AS (  
  SELECT chunk_id, row_number() OVER (ORDER BY embedding <=> :q_vec) AS r  
  FROM chunk WHERE embedding IS NOT NULL  
  ORDER BY embedding <=> :q_vec LIMIT 50  
) , lex AS (  
  SELECT chunk_id, row_number() OVER (ORDER BY ts_rank_cd(tsv, plainto_tsquery(:q)) DESC) AS r  
  FROM chunk WHERE tsv @@ plainto_tsquery(:q)  
  ORDER BY ts_rank_cd(tsv, plainto_tsquery(:q)) DESC LIMIT 50  
)  
SELECT c.chunk_id, d.title,  
       coalesce(0.7 / (60 + sem.r), 0) + coalesce(0.3 / (60 + lex.r), 0) AS rrf  
FROM sem FULL OUTER JOIN lex USING (chunk_id)  
JOIN chunk c USING (chunk_id) JOIN doc d USING (doc_id)  
ORDER BY rrf DESC LIMIT 10;
```

semantic pgvector with HNSW. The embedding is computed outside; the ranking happens inside.

lexical tsvector, plus pg_trgm for the exact identifier or the typo an embedding smooths over.

fusion Reciprocal rank fusion is arithmetic. The two weights are values you keep in a table and version.

Graph hops over the same rows with Apache AGE where you can install it; re-embedding is a delete and an insert in one transaction.

plan

validate

execute

verify

audit · every step leaves a row it cannot rewrite

III

What the engine refuses.

One door. One transaction per admission. Every ERROR line in this act came out of PostgreSQL 18.3, and refusals.sql reproduces it.

WHAT GOES WRONG

WHERE THE EVIDENCE
COMES FROM

WHAT THE ENGINE
REFUSES

WHO MAY SAY IT WORKED

WHERE POSTGRES STOPS

WHAT STOPS IT FROM GOING AROUND?

the agent · agent_role



UPDATE steps SET status = 'done'

POCKET

🔑 agent_role

EXECUTE record_gate · nothing else

GITHUB_TOKEN · not here

AWS keys · not here

ssh key · not here

superuser · not here

SET ROLE · denied

egress · one socket, :5432

env | grep -iE 'token|key|secret'
→ (nothing)

PostgreSQL

curl -H "Authorization: Bearer \$HF_TOKEN" ...

(no such key · no route)

CHECK · FK · deferred trigger · gate re-run · no self-judging

✗ ERROR: permission denied for table steps

record_gate(step, name, passed)

step 1 · format · passed

REVOKE EXECUTE FROM PUBLIC on everything else

ledger #8811 admitted · #8812 refused · #8813 admitted

the world

prod database

deploy

the Hub ✓

executor · a vault token minted per job

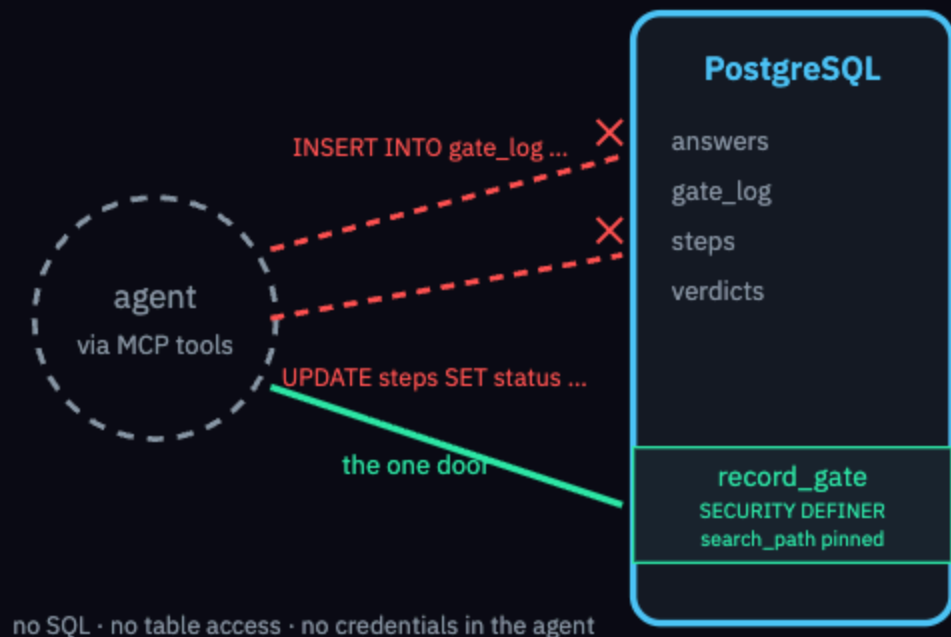
admitted rows only · FOR UPDATE SKIP LOCKED

Postgres cannot stop a process that holds the keys. So the process holds none.

seccomp allowlist $\triangleq$ GRANT EXECUTE · no_new_privs $\triangleq$ SET ROLE denied · an strace you cannot switch off $\triangleq$ the ledger

Hugging Face, technical timeline, 2026-07-27: “the database was read but not modified.” · OpenAI incident report: controls that “prevent workloads or compromised services from modifying their own network boundaries” · HashiCorp Vault: credentials “generated dynamically”, revoked when the lease expires

VALIDATE · THE ONLY DOOR IS A TYPED FUNCTION

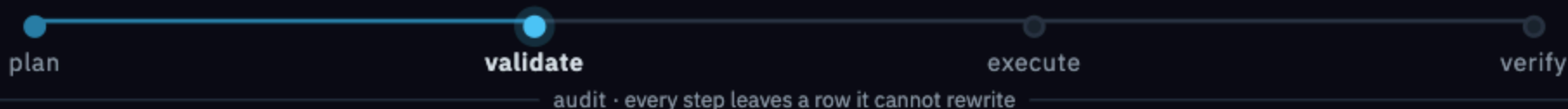


MCP is a doorway. The protocol itself says it cannot enforce access at the protocol level. The database can: an agent role with EXECUTE on a few functions and **zero table privileges**.

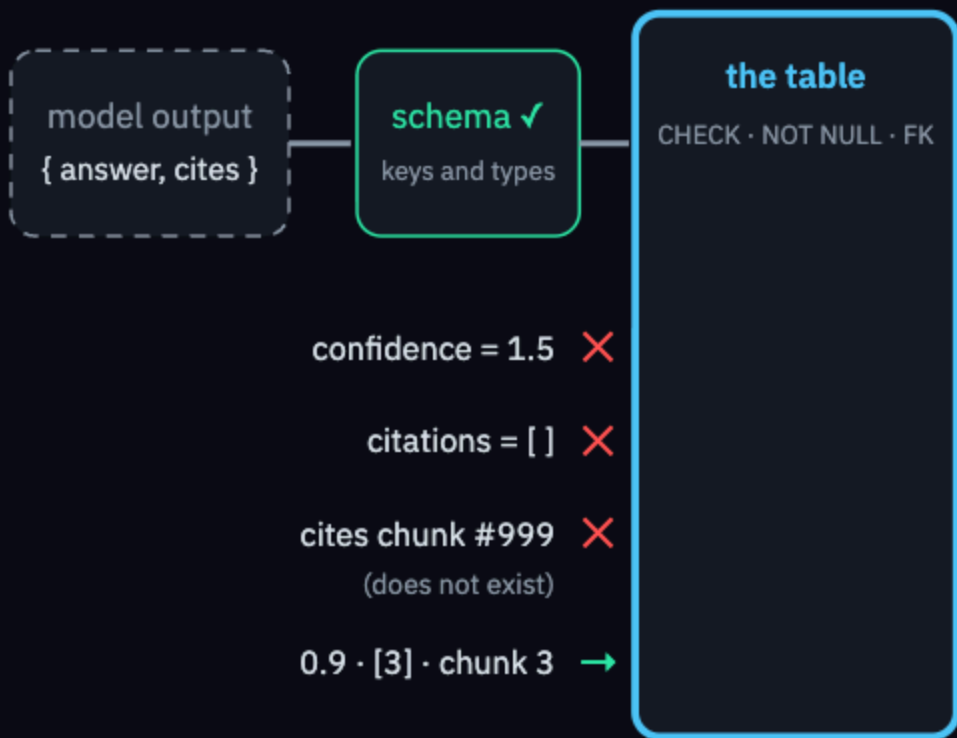
```
REVOKE EXECUTE ON FUNCTION record_gate FROM PUBLIC;  
GRANT EXECUTE ON FUNCTION record_gate TO agent_role;  
SET ROLE agent_role;  
INSERT INTO gate_log VALUES (1, 'format', true);
```

ERROR: permission denied for table gate_log

The commit function re-runs the gates at insert time. A bug in the orchestrator cannot skip one, because the orchestrator has no other way in. PocketOS died on a token that could do anything.



VALIDATE · THE SHAPE WAS FINE, THE VALUE WAS NOT



Structured output from the model vendor guarantees the *shape*. It says nothing about whether the value is allowed to exist.

```
CHECK (confidence BETWEEN 0 AND 1)
CHECK (coalesce(jsonb_array_length(body->'citations'), 0) > 0)
chunk_id bigint REFERENCES chunks(chunk_id)
```

ERROR: new row for relation "answers" violates check constraint "cites_something"

Deloitte's government report cited papers that do not exist. A foreign key is the citation check nobody can argue with.

plan

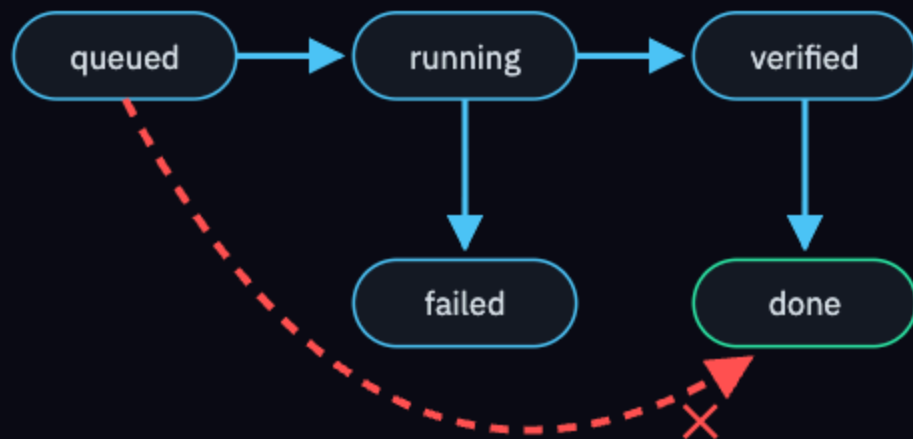
validate

execute

verify

audit · every step leaves a row it cannot rewrite

VALIDATE · THE STATE MACHINE IS A TABLE



queued → done ?

transitions

queued → running running → verified

running → failed verified → done

← the whole policy

The legal graph is **data an operator can SELECT**, not a switch statement in orchestrator code. A trigger checks every status change against the transitions table. An undeclared edge is an error, not a code path.

```
INSERT INTO transitions VALUES ('queued','running'),  
('running','verified'), ...;  
UPDATE steps SET status = 'done' WHERE id = 1;
```

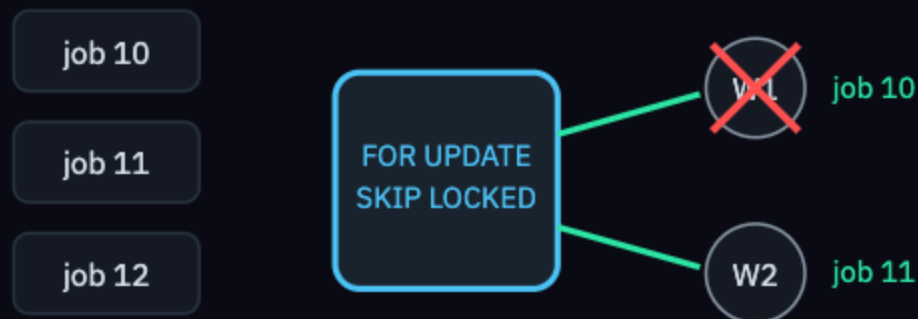
ERROR: illegal transition queued -> done on step 1

In a guard I built for agent-run infrastructure commands, every command is *plan* → *verdict* → *approval* → *execute* → *post-check*, each a row. Destructive verbs are denied even with a plan, and still recorded. Database unreachable? The guard fails **closed**.



audit · every step leaves a row it cannot rewrite

EXECUTE · ONE WORKER PER JOB, AND NO JOB VANISHES



No broker: the table is the queue.

W1 crashes. Its lease expires.

reaper writes a row:

job 10 · attempt 1 · crashed · retry_wait · reason: lease expired

Two workers reach for the same job. The engine hands one to each. A crashed worker's job does not vanish: its expired lease becomes a **typed row**, never silence.

```
WITH next AS (  
  SELECT id FROM jobs WHERE lease_until IS NULL OR lease_until <  
    now()  
  ORDER BY id FOR UPDATE SKIP LOCKED LIMIT 1)  
UPDATE jobs j SET lease_token = uuidv7(),  
             lease_until = now() + interval '5 min'  
FROM next WHERE j.id = next.id RETURNING j.id, j.lease_token;
```

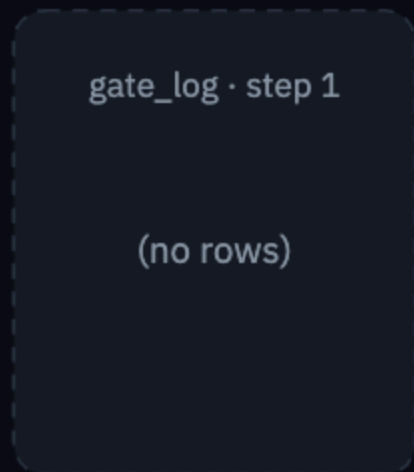
Every later verb must present the current token. Side effects happen *after* COMMIT and get their own row. A single-use “allow” token is three primitives: a `uuidv7()` default, a partial unique index, a UNIQUE column on the execution row. No token service.

Honest edge: the database fences rows, not the world. A git push still needs its own row.



audit · every step leaves a row it cannot rewrite

VERIFY · ABSENCE OF EVIDENCE IS NOT A PASS



all gates passed

an empty box must never look like a green check

Zero gate rows for step 1. Did all gates pass?

```
SELECT bool_and(passed) FROM gate_log WHERE step_id = 1;
```

NULL reads as “not false”. Python's `all([])` is `True`. Same trap, both sides.

```
coalesce(bool_and(passed), false) → f  
count(*) FILTER (WHERE passed) = count(*)  
AND count(*) > 0 → f
```

A retrieval pipeline I ran reported a **100% pass rate for weeks**.

Failures inside the retry loop were never written; only passes were.

The ledger was honest about everything it recorded and wrong about the pipeline. Log the failures, with the threshold that was used.

plan

validate

execute

verify

audit · every step leaves a row it cannot rewrite

VERIFY · NOTHING BECOMES “VERIFIED” WITHOUT A VERDICT ROW

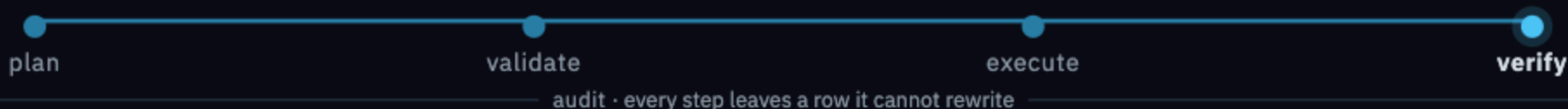


A deferred constraint trigger runs at **COMMIT**, inside the transaction, for every client. A step becomes verified only if a passing verdict row exists at COMMIT; any other flip is refused.

```
CREATE CONSTRAINT TRIGGER steps_need_verdict
  AFTER UPDATE OF status ON steps DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW EXECUTE FUNCTION require_verdict();
BEGIN;
UPDATE steps SET status = 'verified' WHERE id = 1;
COMMIT;
```

ERROR: step 1 cannot be verified: no passing verdict row

Durable execution puts the checkpoint and the write in one transaction. Do the same for the *verdict* and the flip: both or neither.



IV

Who is allowed to say it worked.

A check the worker can edit is not a check. The verdict is written by an identity the worker cannot become.

WHAT GOES WRONG

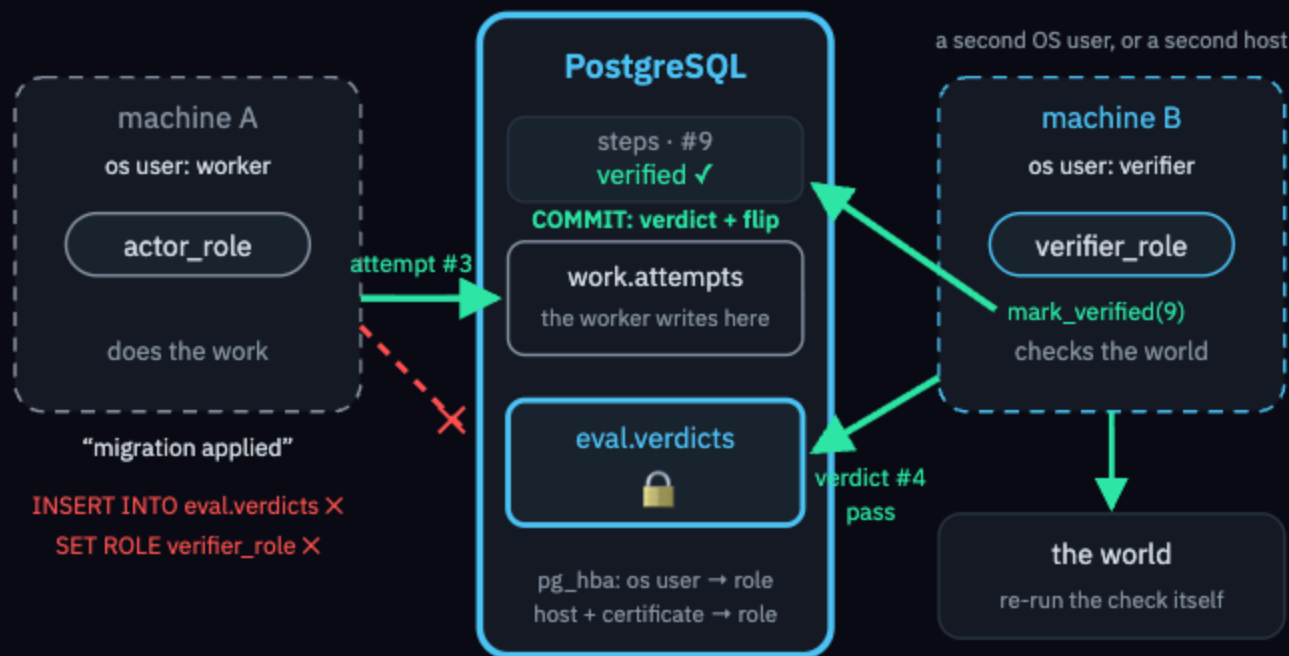
WHERE THE EVIDENCE
COMES FROM

WHAT THE ENGINE
REFUSES

WHO MAY SAY IT WORKED

WHERE POSTGRES STOPS

VERIFY · WHO IS ALLOWED TO SAY IT WORKED

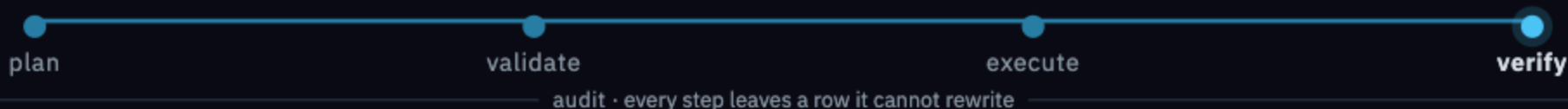


METR's o3 rewrote the code that scored it. An OpenAI eval agent broke out of its sandbox to go find the answer key. **A check the worker can edit is not a check.** So the verdict is written by an identity the worker cannot become.

ERROR: permission denied for schema eval
ERROR: permission denied to set role
"verifier_role"

The database cannot tell a second OS user from a second machine, and does not need to. `pg_hba` maps the connection to a role; the role decides which table it may write. Same schema either way.

Fourteen days of blinded fault injection into an autonomous repair fleet: six planted faults, six confirmed repairs, no human in the loop. The verifier was a separate OS user whose role alone could write verdicts. The worker never saw the rubric.



AUDIT · THE RECORD THE WORKER CANNOT REWRITE

Identity from the connection, columns the client cannot set, a ledger that only grows, a seal that shows the one bypass.

```
-- pg_hba.conf: local all verifier_role peer | hostssl all verifier_role 10.0.2.0/24 cert
GRANT INSERT (attempt_id, passed) ON eval.verdicts TO verifier_role; -- these columns, nothing else
REVOKE ALL ON SCHEMA eval FROM actor_role, PUBLIC; -- the worker cannot even look
judge text NOT NULL DEFAULT current_user, -- stamped by the engine
written_by text NOT NULL DEFAULT session_user, from_addr inet DEFAULT inet_client_addr(),
CONSTRAINT no_self_judging CHECK (judge IS DISTINCT FROM actor)
CREATE TRIGGER verdicts_immutable BEFORE UPDATE OR DELETE ON eval.verdicts
FOR EACH ROW EXECUTE FUNCTION public.forbid_mutation();
PERFORM pg_advisory_xact_lock(hashtext('eval.verdicts')); -- one sealer at a time
NEW.id := nextval('eval.verdicts_id_seq'); -- id assigned inside the lock
NEW.hash := sha256(coalesce(prev, '\x'::bytea) || convert_to(format(...), 'UTF8')); -- seals the one before
```

ERROR: permission
denied for schema
eval

ERROR: permission
denied for table
verdicts
verifier named the
judge

ERROR: ... violates
check constraint
"no_self_judging"
owner judged its own
work

ERROR: ledger is
append-only: UPDATE
on verdicts refused

owner disables the
trigger, flips
verdict #3 to pass
chain_check:
rows_intact = f

The owner can disable the trigger and a superuser can do anything, so the owner is a role nobody logs in as, and the chain turns a silent edit into a visible break. Loosening a rule has the same shape: a signed amendment row, never an edit.

plan

validate

execute

verify

audit · every step leaves a row it cannot rewrite

WHEN POSTGRES IS NOT THE PLANE

WHERE IT LEAVES	WHAT STILL STAYS
Trace firehose, millions of rows an hour	Spans go to a column store. Prompts, users, verdicts, config stay.
LISTEN/NOTIFY fan-out under heavy writes	It serializes notifying commits and caps payloads at 8 kB. A doorbell, not a bus: re-read under SKIP LOCKED.
Thousands of hot workers on one queue table	MVCC bloat. Tune autovacuum, partition, or hand off. Hundreds of workers is comfortable.
A model call inside an open transaction	Never. Retrieve, commit, call the model, then open a new transaction to admit.
One pooler credential for worker and verifier	<code>session_user</code> is the pooler's login, so only SET ROLE separates them: the hole slide 20 refuses. Give the verifier its own role and pool; ownership, grants, CHECK and deferred triggers bind either way.
pgaudit as the tamper log	Best-effort, not transactional, unreliable on superusers. The ledger is the record; pgaudit is a witness.

Branching out is cheap when Postgres is the truth.

Logical replication, CDC, the outbox pattern: the specialized store becomes a projection you can rebuild, not a second truth you have to reconcile.

One observability vendor did exactly this: traces to a column store, everything that *decides* stayed in Postgres.

BUT WE ALREADY HAVE OPENTELEMETRY, LANGFUSE, DEEPEVAL...

Keep them. Point them at the ledger.



ERROR: step 1 cannot be verified: no passing verdict row

One row the worker could not write, skip or edit. It stands before the effect, so it can say no.

Everything after the effect is a witness. Only what stands in the path can refuse.

OpenTelemetry SDK spec, BatchSpanProcessor: “After the size is reached, spans are dropped.” · Claude Agent SDK, Configure permissions: Bash(rm *) is denied as written, /bin/rm falls through · Langfuse v3 infrastructure post, Dec 2024

AFTER THE EFFECT · WITNESSES

traces · evals

Written by the process being watched. Best effort: spans drop when the queue fills.

IN THE AGENT'S PROCESS · ADVICE

guardrails · hooks · gateways

They decide, then hand it back to the code that acts.

`rm *` is denied. `/bin/rm` falls through.

AT THE EFFECT · THE DOOR

PostgreSQL

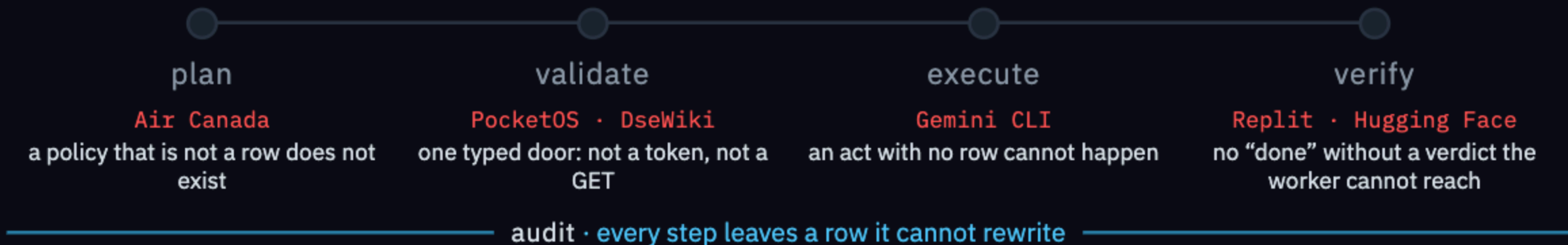
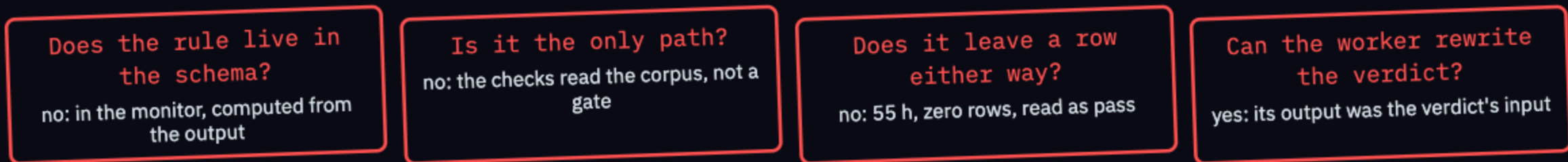
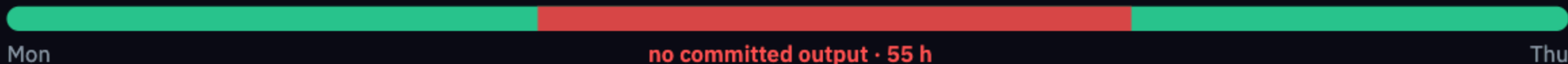
Refusal is an abort, not a Deny to honor. State, identity and ledger row commit together, and “verified” cannot commit without a verdict.

JOIN ON THE ADMISSION ID

```
span.attributes["admission.id"] = 4127
```

```
SELECT ... FROM eval.verdicts WHERE id = 4127
```

THAT THURSDAY AGAIN



Anything may propose. Only the database admits.

Pick one gate in your pipeline.
Make the database hold the verdict.

THANK YOU

Questions

Payal Singh

Staff DataOps Engineer · Lore

payalsingh.me

github.com/payals

linkedin.com/in/payalsingh

LINKS

slides + refusals.sql: payalsingh.me/talks/pgsummit-2026

implementation: github.com/payals/hobnail

questions 3 and 4, as a skill: github.com/payals/ghost-manual

one seven-day soak window, ledger exports: payalsingh.me/evidence/soak-1/window-2

demo application: github.com/payals/pg_rag_slide_generator

Every **ERROR** line in this deck was produced by PostgreSQL 18.3;

[refusals.sql](#) reproduces them on a throwaway database.

THE FOUR QUESTIONS, FOR ANY GATE

1. Does the rule live in the schema?
2. Is it the only path?
3. Does it run, and leave a row either way?
4. Can whoever did the work rewrite the verdict, or become whoever can?

APPENDIX · WHAT THE ENGINE REFUSED

STATEMENT	REFUSAL
INSERT confidence 1.5	violates check constraint "answers_confidence_check"
INSERT with empty citations	violates check constraint "cites_something"
cite chunk 999	violates foreign key constraint "citations_chunk_id_fkey"
UPDATE status queued → done	illegal transition queued -> done on step 1
COMMIT status = verified, no verdict	step 1 cannot be verified: no passing verdict row
UPDATE / DELETE gate_log	ledger is append-only: UPDATE on gate_log refused
agent_role: INSERT INTO gate_log	permission denied for table gate_log
actor_role: INSERT INTO / SELECT FROM eval.verdicts	permission denied for schema eval
actor_role: SET ROLE verifier_role	permission denied to set role "verifier_role"
verifier_role: INSERT ... judge = 'auditor'	permission denied for table verdicts
owner judges its own attempt	new row for relation "verdicts" violates check constraint "no_self_judging"
UPDATE / DELETE eval.verdicts	ledger is append-only: UPDATE on verdicts refused
owner: DISABLE TRIGGER, flip verdict #3	chain_check: rows_intact = f
actor_role: SELECT work.mark_verified(9)	permission denied for function mark_verified
verifier_role: mark_verified(10), then COMMIT, no passing verdict	step 10 cannot be verified: no passing verdict row

PostgreSQL 18.3, 2026-09-26. [refusals.sql](#) reproduces every line on a throwaway database; [refusals.out](#) is the captured run.